

Python Learning Roadmap – Week 1

Author: Tejesh

Goal: Building strong Python fundamentals for Data Science and AI/Machine Learning.

Duration: Structured Weekly Learning Plan

Introduction

This document outlines my structured learning roadmap for **Python, Data Science, and AI/Machine Learning**.

The goal of this roadmap is to build strong foundations in Python programming and gradually progress toward advanced topics such as **Data Analysis, Machine Learning, and AI**.

I am following a **consistent weekly learning plan**, documenting the topics I learn each week to track my progress and maintain discipline in my journey toward becoming an **AI/Machine Learning Engineer / Data Scientist**.

This roadmap is designed to focus on:

- Python programming fundamentals
- Data manipulation and analysis
- Maths that is used in Data Science and AI
- Machine learning concepts
- Artificial intelligence concepts
- Practical project implementation

By sharing this roadmap publicly, I aim to **stay accountable, track my growth, and connect with professionals in the AI/Data Science community**.

Week 1: Python Fundamentals

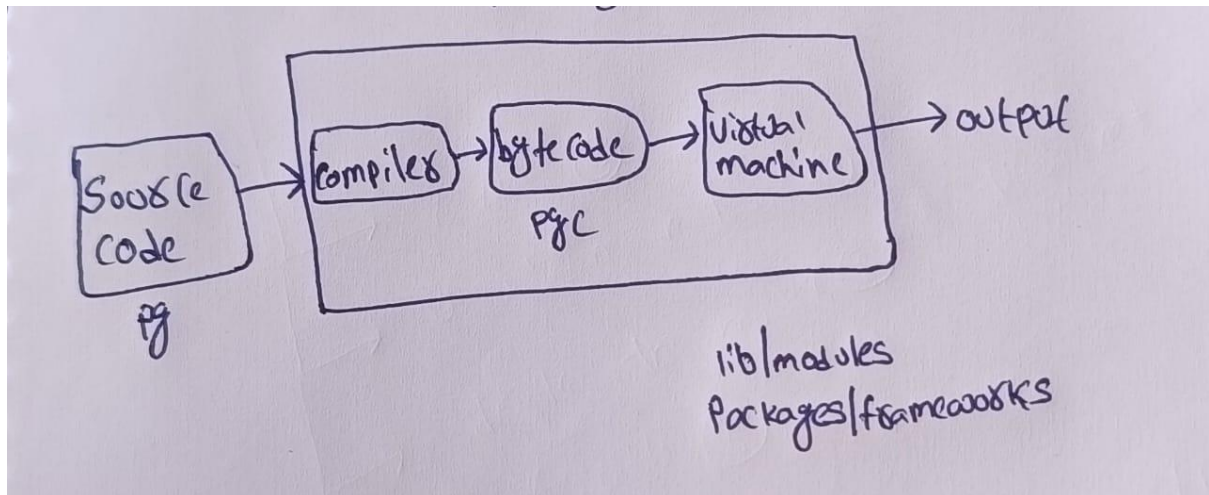
Topics Covered

- Introduction to Python
- Python syntax and structure
- Comments
- Variables and data types
- Operators
- Conditional statements (if, elif, else)
- Logical operators
- Basic problem solving

Introduction to Python

- Python is an object-oriented, high-level, and interpreted programming language.
- It is a user-friendly language compared to all other programming languages
- Python is a large community and it has vast number of lib, packages, modules, frameworks

Let's see how Python works



Source code: our python code that we saved in “.py”

It internally converts code → bytecode (.pyc) → executed by Python Virtual Machine (PVM)

Interpreter: it will check the code line by line

Bytecode is an **intermediate representation**, not machine code

It is executed by the **Python Virtual Machine**

Virtual machine: executes bytecode instructions and runs the program

Python Syntax and structure

In Python **we should follow indentations** we need to give correct space then only our code will run and execute for example

input() – takes user input

print() – displays output

Python uses indentation instead of curly braces {} to define code blocks

```
a = "hi"
b = "hello"
c = a + b
print(c)
if c == "hellohi":
    print("c is equal to hellohi")
else:
    print("c is not equal to hellohi")
```

✓ 0.0s

```
hihello
c is not equal to hellohi
```

In the above image if you see we gave the correct indentation so the code executed successfully

```
a = "hi"
b = "hello"
c = a + b
print(c)
if c == "hellohi":
print("c is equal to hellohi")
else:
print("c is not equal to hellohi")
```

✗ 0.0s

```
Cell In[5], line 6
    print("c is equal to hellohi")
    ^
IndentationError: expected an indented block after 'if' statement on line 5
```

If you see in the last line of error, we got an indentation error because we didn't have any space after our if conditions so we got an error

Comments in Python

Comments are used to explain code for our understanding and comments won't run

There are 2 types

1. Single line comments
2. Multi-line comments

is used in single line comments

This is used in multi-line comments

''' or """

.....

...

“” or “””

Variables in Python

It is used to store values.

Variable = value

Syntax

Variable = values

X = variable, 10 (or) “hello” = value

X = 10

Y = “hello”

Rules for variables

1. We shouldn't start with digit (or) symbol
2. We should always start with letters (or) underscore
3. It is case-sensitive

12ab = 10 ✗

Ab12 = 10 ✓

_ab = 10 ✓

Data Types in Python

1. Numeric Data Types

These data types are used to store numerical values.

- **Integer (int)**
Represents whole numbers without any decimal point. It can store both positive and negative values.
- **Float (float)**
Represents numbers with decimal points. It is used for precise calculations involving fractions or real numbers.
- **Complex (complex)**
Represents numbers with a real and an imaginary part. These are mainly used in advanced mathematical and scientific computations.

Where it is used:

Used in calculations, data analysis, scientific computing, and financial applications.

2. Sequence Data Types

These data types store a collection of ordered elements.

- **String (str)**
Represents a sequence of characters. Strings are used to store and manipulate text.
- **List (list)**
An ordered and mutable collection of elements. It allows modification such as adding, removing, or changing elements.
- **Tuple (tuple)**
An ordered but immutable collection of elements. Once created, its values cannot be changed.

Where it is used:

Used for storing collections of data, text processing, and managing ordered datasets.

3. Set Data Types

- **Set (set)**
An unordered collection of unique elements. It does not allow duplicate values.
- **Frozen Set (frozenset)**
Similar to a set but immutable. Once created, elements cannot be modified.

Where it is used:

Used in removing duplicates, performing mathematical set operations, and handling unique data collections.

4. Mapping Data Type

- **Dictionary (dict)**
Stores data in key-value pairs. Each key is unique and is used to access its corresponding value.

Where it is used:

Used in storing structured data, databases, APIs, and configurations where data is mapped using keys.

5. Boolean Data Type

- **Boolean (bool)**
Represents logical values: True or False.

Where it is used:

Used in decision-making, conditional statements, and logical operations.

6. Binary Data Types

These data types are used to handle binary (byte-level) data.

- **Bytes (bytes)**
Represents immutable sequences of bytes.
- **Bytearray (bytearray)**
Represents mutable sequences of bytes.
- **Memoryview (memoryview)**
Provides a way to access internal data of objects without copying it.

Where it is used:

Used in file handling, network communication, and working with binary data such as images or audio files.

7. None Data Type

- **NoneType (None)**
Represents the absence of a value or a null value.

Where it is used:

Used to indicate no value, default values, or when a variable is intentionally left empty.

Operator's in Python

1. Logical Operators

Operators: and, or, not

These operators are used to combine or modify conditions.

and → Returns true only when all conditions are true

or → Returns true if at least one condition is true

not → Reverses the result of a condition (true becomes false, false becomes true)

Where it is used:

Used mainly in decision-making, conditional statements, loops, and filtering logic in programs.

2. Comparison Operators

Operators: >, <, >=, <=, ==, !=

These operators are used to compare two values.

> → Checks if one value is greater than another

< → Checks if one value is less than another

<= → Checks if one value is less than or equal to another

>= → greater than or equal
= → assignment operator (NOT comparison)
== → Checks if two values are equal
!= → Checks if two values are not equal

The result of comparison is always True or False.

Where it is used:

Used in conditional statements, loops, validations, and decision-making processes.

3. Arithmetic Operators

Operators: +, -, *, /, %, //, **

These operators are used to perform mathematical calculations.

+ → Addition

- → Subtraction

* → Multiplication

/ → Division (gives decimal result)

% → Modulus (gives remainder)

// → Floor division (gives integer result without decimals)

** → Exponentiation (power)

Where it is used:

Used in calculations, formulas, financial computations, data analysis, and algorithm design.

4. Assignment Operators

Operators: =, +=, -=, *=, /=

These operators are used to assign values to variables.

= → Assigns a value

+= → Adds and assigns

-= → Subtracts and assigns

*= → Multiplies and assigns

/= → Divides and assigns

Where it is used:

Used for storing values, updating variables, counters, accumulators, and iterative computations.

5. Membership Operators

Operators: in, not in

These operators are used to check whether a value exists in a collection such as a list, string, or tuple.

in → Returns true if the value is present

not in → Returns true if the value is not present

Where it is used:

Used in searching elements, validation, filtering data, and checking existence in datasets or collections.

6. Identity Operators

Operators: is, is not

These operators are used to check whether two variables refer to the same memory location (object).

is → True if both variables refer to the same object

is not → True if they refer to different objects

Important difference:

Equality (==) checks value

Identity (is) checks memory location

Where it is used:

Used in object comparison, memory reference checking, debugging, and working with mutable and immutable objects.

7. Bitwise Operators

Operators: &, |, ^, ~, <<, >>

These operators work on binary (bit-level) values.

& → AND (both bits must be 1)

| → OR (at least one bit is 1)

^ → XOR (only one bit is 1)

~ → NOT (inverts bits)

<< → Left shift (moves bits left)

>> → Right shift (moves bits right)

Where it is used:

Used in low-level programming, performance optimization, system programming, embedded systems, and cryptography-related applications.

Control Statements in Python

Conditional statements are used to control the code

There are 3 types

- Conditional/ decision making statements
- Looping/ iteration statements
- Jumping Statements

Conditional Statements

This will decide the statements in Python

It has 3 types

- If
- Elif
- Else

If

It will check whether the given condition is true or false if our condition is true then it will print our true condition

Elif

If our condition 1 is false then we'll give another condition in elif is this condition is true then it will print this condition

Else

If the given all conditions are false then the else will print here, we don't need any condition

Syntax

if condition1:

Print("output1")

elif condition2:

print("output2")

else:

print("output3")

Note: All the coding examples for the topics mentioned above are available in my GitHub repository—feel free to check it out for complete reference implementations.

Looping Statements

For loops

It is used to execute a block of code within a number of times, you iterate over a range or string or sequence

In for loops we use range()

The range() function in Python generates a sequence of integers and is most commonly used in loops

continue → skips current iteration

break → stops the loop completely

1st Continue

In range(1, 11), if we don't want to print 8, we use continue.

```
#for loop
for i in range(1,11):
    if i == 8:
        continue
    print(i)
```

✓ 0.0s

1
2
3
4
5
6
7
9
10

2nd Break

If we want only 1 to 5 then we'll use break operation

```
#break
for i in range(1,11):
    if i == 6:
        break
    print(i)
```

✓ 0.0s

1
2
3
4
5

While loops

It executes some code WHILE some conditions remain true

```
#while loop
name = input("Enter your name: ")

while name == "":
    print("you did not enter your name")
    name = input("Enter your name: ")

print(f"Hello {name}")
```

✓ 9.3s

you did not enter your name
you did not enter your name
you did not enter your name
you did not enter your name
Hello Bro

It will iterate until the while statement is false

Basic Problem Solving

In my GitHub I've posted everything click on [GitHub profile](#) to see the coding questions and answers

Summary

Week 1 Summary – Python Fundamentals

- In Week 1, I focused on building a strong foundation in Python programming. I learned the core concepts required to start writing basic programs and understanding how Python works internally.
- I began with an introduction to Python, including its features such as simplicity, readability, and its wide range of applications in data science and artificial intelligence. I also understood the basic working of Python, including source code, bytecode, and the role of the Python Virtual Machine.
- I covered Python syntax and structure, with special emphasis on indentation, which is essential for defining code blocks. I practiced writing clean and properly formatted code.
- I learned about variables and different data types in Python, including numeric, sequence, set, mapping, boolean, binary, and None types. This helped me understand how data is stored and manipulated in Python.
- I explored various operators such as arithmetic, comparison, logical, assignment, membership, identity, and bitwise operators, and understood how they are used in different scenarios.
- I studied control statements, including conditional statements (if, elif, else), and learned how to make decisions in programs. I also practiced looping concepts using for and while loops, along with break and continue statements.
- Overall, this week helped me build a strong base in Python fundamentals, improve my logical thinking, and gain confidence in writing basic programs.

In the upcoming week, we will focus on data structures, including their concepts, operations, and practical usage in Python.